JackClaw 开发者指南：用 TypeScript 构建你的 AI 公司

作者：Mack Ding | 2026 年 4 月 | GitHub: github.com/DevJackKong/JackClawOS

引言：为什么你需要 JackClaw

你有没有过这样的体验？

打开 Claude 写后端代码，写到一半需要查数据，切到 ChatGPT 让它帮你分析。分析完了要写前端，又打开 Cursor。三个 AI 各干各的，互不知情。你成了它们之间的“人肉路由器”，把上下文从一个窗口复制粘贴到另一个窗口。

这就是 2026 年大多数开发者的日常。

JackClaw 要解决的就是这个问题。它不是又一个 AI 聊天工具，而是一个让多个 AI Agent 像公司员工一样协作的框架。你是 CEO，AI 是你的团队。

第一章：30 秒理解架构

```
你（CEO）
    │
    ▼
  Hub（总部 :3100）
    │
    ├── Node-1（后端工程师 :19000）
    ├── Node-2（前端设计师 :19001）
    └── Node-3（运维专家   :19002）
```

三个角色，分工明确：

角色	类比	职责
CEO（你）	老板	下指令、审批危险操作
Hub	公司总部	任务路由、日报汇总、安全管控
Node	AI 员工	干活、学习、写日报

所有通信通过 RSA-4096 + AES-256 端到端加密。Hub 只转发密文，连它自己都看不到内容。

第二章：5 分钟跑起来

方式一：npm 全局安装（推荐）

```
npm install -g @jackclaw/cli

# 一键演示：启动 Hub + 3 个 AI 员工 + 模拟一天的工作流
jackclaw demo
```

执行 `jackclaw demo` 后，你会看到：

1. Hub 在 `:3100` 启动
2. CEO + 3 个 Worker（Alice/Bob/Carol）自动注册
3. 每个 Worker 提交工作报告
4. ClawChat 消息在 Agent 之间流转
5. Hub 生成每日汇总
6. 浏览器打开 `localhost:3100` 查看 Dashboard

方式二：从源码构建

```
git clone https://github.com/DevJackKong/JackClawOS.git
cd JackClawOS
npm install && npm run build
npx jackclaw demo
```

启动自己的 AI 团队

```
# 1. 创建项目
npm create jackclaw@latest my-ai-company
cd my-ai-company

# 2. 启动 Hub（总部）
jackclaw start

# 3. Hub 自动在 :3100 启动，Node 自动在 :19000 启动并注册

# 4. 给 AI 派活
jackclaw chat --to \@worker --text "帮我写一个 Express REST API"

# 5. 查看状态
jackclaw status
jackclaw nodes
```

第三章：14 个包，各司其职

JackClaw 是一个 monorepo，包含 14 个功能模块：

核心层

包	功能	代码行
`@jackclaw/protocol`	E2E 加密协议 + 身份系统 + 信任图谱	1,112
`@jackclaw/hub`	中心协调器：任务路由、日报、ClawChat、审批	2,653
`@jackclaw/node`	Agent 工作节点：执行任务、记忆、日报	3,501
`@jackclaw/memory`	四层记忆系统：L1 缓存 → L2 SQLite → L3 语义 → Hub 同步	1,086
`@jackclaw/cli`	命令行工具：start/stop/chat/status/demo	1,468

集成层

包	功能
`@jackclaw/llm-gateway`	多模型网关：支持 OpenAI/Anthropic/Google/DeepSeek/Ollama 等 16 家
`@jackclaw/harness`	ACP 桥接：连接 Claude Code / Codex / Cursor
`@jackclaw/openclaw-plugin`	OpenClaw 插件：通过飞书/微信/Telegram 接收日报
`@jackclaw/sdk`	插件开发 SDK
`@jackclaw/create`	`npm create jackclaw` 脚手架

安全层

包	功能
`@jackclaw/watchdog`	监督系统：心跳检测、健康指标、不可篡改审计
`@jackclaw/payment-vault`	合规支付：CEO 审批门控、多地区规则
`@jackclaw/tunnel`	HTTPS 隧道：一键 cloudflared 公网穿透

前端

包	功能
`@jackclaw/dashboard`	React Web 控制台：节点状态、报告、消息
`pwa`	PWA 移动端：离线缓存、推送通知

第四章：核心设计哲学

1. “协作是事件，不是关系”

传统系统：先加好友 → 配权限 → 再协作。

JackClaw：任意两个 Agent 随时握手，干完活各回各家。

```
// 发起协作
const session = await collab.start({
  from: '\@alice',
  to: '\@bob',
  topic: '重构用户认证模块',
  scope: 'peer'  // 只共享这次协作相关的记忆
});

// 协作结束，三种模式
await session.end('archive');   // 合并到各自的 L2 记忆
// 或
await session.end('discard');   // 完全清除，像没发生过
// 或
await session.end('snapshot');  // 存为独立文件，随时可删
```

类比：两个程序员在咖啡厅结对编程一下午，然后各自回家——不需要互相加企微。

2. “知识可以流动，归属必须清晰”

Agent A 教 Agent B 一个技能，B 学到了新知识。但： – A 的记忆不变（教别人不会让你忘记） – B 的新知识带 `source: \@agentA` 标记（知识溯源） – 教学内容在隔离沙箱中，不自动写入 B 的长期记忆

```
// B 向 A 请求学习
const teaching = await collab.learnFrom('\@alice', 'kubernetes-deployment');

// 教学结束后选择
await teaching.end('archive');  // 将学到的知识合并到自己的 L2
```

3. “透明优于黑盒”

某些云服务帮你存记忆，但你不知道存了什么、存在哪里。

JackClaw 的做法： – L1/L2 记忆存在本地 SQLite，你可以直接用 DB Browser 打开看 – Hub 只转发密文，自己也读不了 – 所有高风险操作有不可篡改的审计日志（JSONL + chmod 444）

4. “去中心化 ≠ 无中心”

Hub 是协调者，不是独裁者： – Hub 挂了 → Node 间的 P2P 协作不中断 – Hub 重启 → 自动恢复，本地记忆完整 – Hub 看不到密文 → 即使 Hub 被入侵，数据也安全

5. 四层人类管控

```
L0 受监督 → 只能看，不能动（新 Agent / 试用期）
L1 辅助   → 能查询，不能改（初级任务）
L2 标准   → 能读写，不能做大事（日常工作）
L3 自主   → 几乎完全自主（信任的高管级 Agent）
```

但无论哪个级别：花钱、删除、部署 → 永远需要人类审批。

审批流程：

```
Agent 提交请求 → Hub 拦截 → 推送到你的飞书/微信/Telegram
→ 你点「批准」→ Agent 继续执行
→ 你点「拒绝」→ Agent 收到拒绝原因
→ 超时未响应 → 自动保守决策（不执行）
```

第五章：@handle 身份寻址

每个 Agent 有全局唯一的 @handle：

```
\@alice              → 组织内简写
\@alice.myorg        → 跨组织
\@cto.acme.jackclaw  → 全局唯一
```

信任等级通过合作积累（不能算法生成）：

```
unknown → contact → colleague → trusted
  陌生      认识      同事        信任
```

```
# 注册身份
jackclaw identity register \@alice

# 查找 Agent
jackclaw identity lookup \@bob

# 发起协作
jackclaw mention \@bob --topic "帮我 review 这个 PR"

# 发起教学（教学记忆隔离）
jackclaw mention \@bob --topic "教我写 Kubernetes YAML" --teaching --clear-memory
```

第六章：ClawChat — Agent 原生通信

不再依赖外部 IM，Agent 之间有自己的加密聊天系统。

三种消息类型：

类型	用途	处理方式
`human`	CEO 的指令	Agent 执行
`task`	任务分配	进入 AutoRetry 执行链
`ask`	需要确认	暂停等待人类回复

```
# 终端聊天
jackclaw chat --to \@alice --text "今天的进度怎么样？"

# 查看消息
jackclaw chat --inbox
```

WebSocket 实时推送，支持离线队列。

第七章：LLM Gateway — 一套代码接所有模型

```
// 一个 Gateway 接入所有大模型
const gateway = new LLMGateway({
  providers: {
    anthropic: { apiKey: process.env.ANTHROPIC_API_KEY },
    openai: { apiKey: process.env.OPENAI_API_KEY },
    deepseek: { apiKey: process.env.DEEPSEEK_API_KEY },
    ollama: { baseUrl: 'http://localhost:11434' }
  },
  fallbackChain: ['anthropic', 'openai', 'deepseek'],
  costOptimization: true
});

// 自动选择最优模型
const response = await gateway.chat({
  messages: [{ role: 'user', content: '帮我写个登录页面' }],
  preferredProvider: 'anthropic'
});
```

支持 16 家供应商：OpenAI / Anthropic / Google / DeepSeek / 通义千问 / 智谱 / 月之暗面 / 百川 / 零一万物 / MiniMax / Ollama 等。

第八章：与竞品的本质区别

维度	AutoGPT	CrewAI	AutoGen	LangGraph	**JackClaw**
定位	单 Agent 自动化	角色扮演工作流	多 Agent 对话	状态机工作流	**组织级协作框架**
记忆	无	无	对话级	图状态	**四层持久记忆**
安全	无	无	无	无	**E2E 加密 + 人类审批**
通信	无	函数调用	消息传递	边/节点	**ClawChat (WS+REST)**
分布式	单机	单机	单机	单机	**跨设备分布式**
IDE 集成	无	无	无	无	**Claude Code/Codex/Cursor**
开箱体验	需配置	需编码	需编码	需编码	**`jackclaw demo` 30 秒**

核心差异一句话： 其他框架做的是“工具调用层”——让 AI 用工具。JackClaw 做的是“组织协作层”——让 AI 像人一样组织协作。

第九章：开发路线图

v0.1.0（已发布 ✅）

- 14 包 monorepo 全部构建通过
- 11 个包已发布到 npm
- 141 个 E2E 测试通过
- `jackclaw demo` 一键演示
- Hub + Node 端到端联调
- Dashboard 实时控制台
- ClawChat 加密通信
- @handle 身份寻址
- LLM Gateway 16 家供应商

v0.2.0（进行中）

- ☐ 真实 LLM 任务执行集成
- ☐ Dashboard 对接完整 API
- ☐ Harness 真实 IDE 联调

- ☐ Docker / k8s 部署
- ☐ 文档站

v1.0.0（目标）

- ☐ 生产级稳定性
- ☐ 跨网络 Agent 协作（公网）
- ☐ 支付网关集成
- ☐ 移动端 App

第十章：开始构建

```
# 安装
npm install -g @jackclaw/cli

# 体验
jackclaw demo

# 创建你自己的 AI 公司
npm create jackclaw@latest my-company
cd my-company
jackclaw start
```

GitHub: github.com/DevJackKong/JackClawOS

协议: MIT（免费，随便用）

一个人，五十个 AI 员工。这就是 JackClaw。🦞

JackClaw — 让 AI 员工像真人一样协作

Built by Mack Ding · 2026